

Weekly Progress Report

Name: NAGESHWARAN C

Domain: CORE JAVA

Date of Submission: 07-06-2026

Week Ending: 03

I. Overview

Week 3 marked a significant phase of advancement in both project development and Java skill enhancement. The Banking Information System continued to evolve with new features integrated and existing modules improved for robustness and maintainability. Concurrent to project work, deeper exploration of advanced Java concepts was undertaken to strengthen the technical foundation and directly apply learnings to the ongoing project.

II. Achievements

1. Project Development — Banking Information System

- Completed the Account Management module with full support for savings and current account types, including deposit, withdrawal, and balance inquiry operations.
- Improved the Transaction Processing module by introducing a transaction history feature using ArrayList, enabling chronological tracking of all account activities.
- Refactored the Customer Data Handling layer to use HashMap for faster lookup of customer records by account number, improving overall application efficiency.
- Enhanced input validation logic with more comprehensive custom exception classes (e.g., InsufficientFundsException, InvalidAccountException) for cleaner error reporting.
- Implemented a basic Menu-Driven Console Interface to allow users to navigate between banking operations interactively.
- Reviewed and merged team contributions, ensuring consistent coding standards and reducing code duplication across modules.

2. Java Concepts Learned

- **Interfaces and Abstract Classes:** Studied the use of interfaces (e.g., Transactable, Auditable) and abstract classes to define contracts and promote loose coupling in the Banking System's design.
- **Generics:** Learned to apply Java Generics for type-safe collection handling, improving the flexibility and reusability of utility methods in the project.
- **JDBC Basics:** Introduced to Java Database Connectivity — studied how to establish connections, execute queries, and handle ResultSets as groundwork for future database integration.
- **Static and Final Keywords:** Deepened understanding of static utility methods and final constants, applied to define bank-wide configuration values in the project.
- **Iterator and Enhanced For-Loop:** Practiced iterating over Collections using both Iterator and enhanced for-loop patterns for processing transaction records.
- **Java Streams (Introduction):** Got an introductory exposure to Stream API for filtering and summarizing collections, exploring its potential for future use in report generation features.

3. Collaboration and Communication

- Participated in a mid-week team sync to review module integration points and resolve interface mismatches.
- Received and incorporated mentor feedback on class design — particularly on applying the Single Responsibility Principle (SRP) to separate concerns in the account module.
- Shared learning notes on Interfaces and Abstract Classes with team members, contributing to the group's overall Java knowledge base.

III. Challenges

4. Project Complexity

- Integrating the new transaction history feature with the existing module structure required careful refactoring to avoid breaking previously tested functionality.
- Designing a clean interface hierarchy for different account types (savings vs. current) needed multiple iterations to achieve a scalable and readable structure.

5. Applying New Java Concepts

- Understanding when to use an interface versus an abstract class in the project context required additional reading and discussion with the mentor.
- Initial JDBC setup and understanding of the connection lifecycle (open/close/exception handling) was complex and required focused study.

IV. Learning Resources

6. Java Learning Platforms

- Referred to official Oracle Java documentation for Interface, Abstract Class, and Generics specifications.
- Followed structured video tutorials on JDBC fundamentals from Java learning platforms.
- Practiced coding exercises on online Java platforms to reinforce Generics and Stream API concepts.

7. Practical Application

- Applied Interface-based design directly in the Banking Information System to model account behaviors.
- Wrote standalone JDBC test programs to practice database connection handling independently before project integration.

V. Next Week's Goals

8. Project Development

- Integrate JDBC into the Banking Information System to persist customer and transaction data in a relational database.
- Write JUnit test cases to validate all implemented modules and improve overall code coverage.
- Complete the remaining pending features: account closure and fund transfer between accounts.

9. Java Learning Goals

- Deepen understanding of Java Stream API and Lambda Expressions for data processing tasks.
- Explore basic design patterns (e.g., Singleton for database connection, Factory for account creation) to improve project architecture.
- Study SQL fundamentals to support JDBC integration effectively.

VI. Additional Comments

Week 3 has been the most technically enriching week so far. The combination of hands-on project development and deliberate Java concept learning has accelerated both skill growth and project progress. The feedback from mentors has been invaluable in shaping better design decisions. The team continues to work cohesively, and the Banking Information System is taking a clearer and more robust shape with each passing week. The upcoming JDBC integration milestone is something the whole team is looking forward to achieving.

Submitted by: NAGESHWARAN C | Week 03 | 07-06-2026